



Université d'Abomey-Calavi
The Abdus Salam International Center for Theoretical Physics
INSTITUT DE MATHÉMATIQUES ET DE SCIENCES PHYSIQUES



Filière : Master I TIC & Recherche Opérationnelle

Rapport de Projet

INTELLIGENCE ARTIFICIELLE

Prédiction des Tendances Boursières

Séries temporelles financières avec Machine Learning

Cours : Intelligence Artificielle

Dr SENOU Rosaire

(Université d'Abomey-Calavi / IMSP)

Présenté par :

AGNINDOM Easo-hana Olivier

ATIN Caleb

ATONDE Marie-Odette

Notebook : projet_IA_groupe5_v4.ipynb · Application : streamlit_app_v4 · Groupe 5

ANNÉE ACADÉMIQUE : 2025-2026

Résumé

Ce rapport présente la méthodologie, les résultats et l'analyse critique du Projet 5 d'Intelligence Artificielle, dont l'objectif est de prédire la tendance journalière (hausse / baisse) des prix de cinq actifs financiers — **AAPL** (Apple), **MSFT** (Microsoft), **BNP.PA** (BNP Paribas), **BTC-USD** (Bitcoin) et **ETH-USD** (Ethereum) — à l'aide de modèles de classification supervisée. Quatre algorithmes sont comparés : Régression Logistique, Random Forest, XGBoost et SVM. Le pipeline complet comprend l'ingénierie de 32 features techniques stationnaires, une validation croisée temporelle (**TimeSeriesSplit**), une recherche d'hyperparamètres par **GridSearchCV**, des tests de significativité statistique (test binomial, test de McNemar) et une simulation de stratégie de trading avec calcul du Sharpe Ratio comparatif. Les accuracies obtenues (50–55 %) sont cohérentes avec la difficulté intrinsèque de la prédiction de direction de marché, documentée dans la littérature. Le seul cas où la stratégie modèle bat nettement le Buy & Hold est **BTC-USD** avec XGBoost (rendement : 236,1 % vs 156,0 %, Sharpe : 1,43 vs 0,95). Une application Streamlit interactive accompagne le notebook.

Table des matières

1	Contexte et Problématique	4
1.1	Contexte général	4
1.2	Problématique	4
1.3	Objectifs opérationnels	4
1.4	Actifs analysés	5
2	Données et Analyse Exploratoire	5
2.1	Source des données	5
2.2	Variable cible	5
2.3	Caractéristiques statistiques des actifs	6
3	Méthodologie	6
3.1	Vue d'ensemble du pipeline	6
3.2	Ingénierie des features	6
3.2.1	Principe : stationnarité des features	6
3.2.2	Features construites	7
3.3	Split chronologique et normalisation	7
3.4	Recherche d'hyperparamètres (GridSearchCV + TimeSeriesSplit) . .	8
3.5	Modèles comparés	8
3.6	Métriques d'évaluation	9
4	Résultats	9
4.1	Analyse détaillée — AAPL (ticker principal)	9
4.1.1	Benchmark des 4 modèles	9
4.1.2	Tests de significativité statistique	10
4.1.3	Importance des variables	10
4.2	Synthèse globale — 5 tickers	11
4.3	Analyse comparative inter-tickers	11
4.3.1	Volatilité et modèle gagnant	11
4.3.2	Cas remarquable : BTC-USD avec XGBoost	12
4.3.3	Cas remarquable : ETH-USD	12
4.4	Simulation de trading	12
4.4.1	Principe	12
4.4.2	Limites de la simulation	13
5	Application Streamlit	13
6	Discussion et Limites	14
6.1	Limites inhérentes à la prédiction de marché	14

6.2	Limites du projet	14
6.3	Perspectives d'amélioration	15
7	Conclusion	15
A	Structure du projet	15
B	Dépendances Python	16
C	Lancement de l'application	16
D	Versions des corrections apportées	17

1 Contexte et Problématique

1.1 Contexte général

Les marchés financiers génèrent chaque jour des volumes massifs de données structurées (prix OHLCV¹, volumes, carnets d'ordres). Depuis les années 2000, les méthodes d'apprentissage automatique sont appliquées à la prédiction des tendances de marché avec un succès variable : la difficulté fondamentale tient à l'hypothèse d'efficience des marchés (EMH, *Efficient Market Hypothesis*), qui stipule que les prix intègrent instantanément toute l'information disponible, rendant la prédiction systématique quasi impossible au-delà du hasard.

En pratique, des travaux récents montrent qu'il est possible d'extraire un signal modeste mais statistiquement significatif à partir d'indicateurs techniques sur certains actifs et certaines périodes. L'objectif n'est pas de « battre le marché » de façon garantie, mais de construire rigoureusement un système de prédiction, d'en mesurer la performance réelle, et d'en identifier les limites.

1.2 Problématique

Question centrale

Peut-on, à partir des données historiques de prix OHLCV et d'indicateurs techniques dérivés, construire un classificateur supervisé capable de prédire la direction du prix d'un actif financier le lendemain, avec une accuracy statistiquement supérieure au hasard ? Quelle famille de modèles offre le meilleur compromis précision / interprétabilité / vitesse sur des actifs de natures différentes (actions, crypto-monnaies) ?

1.3 Objectifs opérationnels

1. Construire un pipeline ML reproductible, applicable à tout actif disponible via `yfinance`.
2. Comparer quatre modèles (Régression Logistique, Random Forest, XGBoost, SVM) avec des hyperparamètres optimisés et une validation temporellement cohérente.
3. Tester la significativité statistique des résultats (test binomial, McNemar).
4. Simuler une stratégie de trading basée sur les prédictions et la comparer au benchmark Buy & Hold via le Sharpe Ratio annualisé.
5. Déployer le système sous forme d'application Streamlit interactive.

1. Open, High, Low, Close, Volume.

1.4 Actifs analysés

Les cinq tickers retenus couvrent délibérément des classes d'actifs et des régimes de marché différents, afin de tester la robustesse du pipeline :

TABLE 1 – Actifs financiers analysés

Ticker	Actif	Classe	Période	Nb jours
AAPL	Apple Inc.	Action US	2015–2025	2 716
MSFT	Microsoft Corp.	Action US	2015–2025	2 716
BNP.PA	BNP Paribas	Action EU	2015–2025	2 766
BTC-USD	Bitcoin	Crypto-monnaie	2015–2025	3 968
ETH-USD	Ethereum	Crypto-monnaie	2015–2025	2 925

2 Données et Analyse Exploratoire

2.1 Source des données

Les données OHLCV journalières sont téléchargées directement via la bibliothèque Python `yfinance`², qui interroge les API Yahoo Finance. Aucune inscription ni clé d'API n'est requise. La période couvre le 1^{er} janvier 2015 au 31 décembre 2025.

Listing 1 – Téléchargement et aplatissement du MultiIndex

```

1 def load_data(ticker, start=START, end=END):
2     df = yf.download(ticker, start=start, end=end,
3                     auto_adjust=True, progress=False)
4     if isinstance(df.columns, pd.MultiIndex):
5         df.columns = df.columns.get_level_values(0)
6     df.dropna(inplace=True)
7     return df

```

2.2 Variable cible

La variable cible est binaire : elle vaut 1 si le prix de clôture du lendemain est supérieur au prix de clôture du jour courant, 0 sinon.

$$\text{target}_t = \mathbf{1}[\text{Close}_{t+1} > \text{Close}_t]$$

Sur **AAPL**, la distribution est quasi équilibrée : environ 53 % de jours haussiers et 47 % de jours baissiers, ce qui rend l'accuracy fiable comme métrique principale et évite d'avoir recours à du rééchantillonnage (SMOTE, etc.).

2. <https://pypi.org/project/yfinance/>

2.3 Caractéristiques statistiques des actifs

Le tableau 2 présente les caractéristiques statistiques clés de chaque actif, qui conditionnent le comportement des modèles.

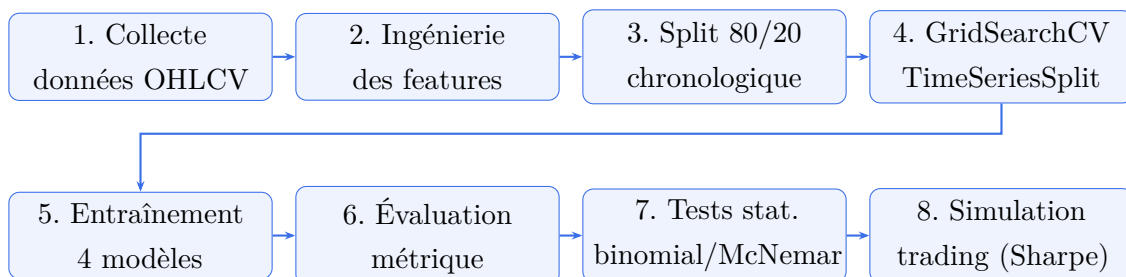
TABLE 2 – Caractéristiques statistiques des actifs (2015–2025)

Ticker	Volatilité quotidienne	Nb observations	Marché
MSFT	1,69 %	2 716	Haussier fort
AAPL	1,82 %	2 716	Haussier fort
BNP.PA	1,97 %	2 766	Neutre/latéral
BTC-USD	3,47 %	3 968	Très haussier volatile
ETH-USD	4,44 %	2 925	Baissier (test)

La volatilité quotidienne est classée par ordre croissant : les actions US présentent la volatilité la plus faible, suivies par BNP Paribas, puis les crypto-monnaies. Sur la période de test (20 % des données les plus récentes), les régimes de marché divergent fortement : **BTC-USD** affiche un marché haussier, **ETH-USD** un marché légèrement baissier, ce qui influencera profondément les performances relatives.

3 Méthodologie

3.1 Vue d'ensemble du pipeline



3.2 Ingénierie des features

3.2.1 Principe : stationnarité des features

Un point méthodologique fondamental distingue ce projet d'une approche naïve : **toutes les features sont construites pour être stationnaires**, c'est-à-dire indépendantes du niveau absolu des prix. Utiliser des prix bruts ou des moyennes mobiles en niveaux absolus introduirait un biais d'échelle : le modèle entraîné sur AAPL à 50 \$ en 2015 ne serait pas cohérent avec AAPL à 230 \$ en 2025. Chaque indicateur est donc exprimé en terme relatif ou différencé.

3.2.2 Features construites

Le pipeline `build_features` produit 32 features réparties en six catégories :

TABLE 3 – Features techniques utilisées

Catégorie	Features	Justification
Rendement	<code>daily_return</code>	Mesure de momentum
Tendance (SMA)	<code>price_vs_SMA5/20/50</code> , <code>SMA_cross</code>	Positionnement relatif, stationnaire
MACD	<code>MACD</code> , <code>MACD_signal</code> , <code>MACD_hist</code>	Momentum court/moyen terme, normalisé par le prix
RSI (14 jours)	<code>RSI_14</code>	Sur-achat / sur-vente
Bollinger	<code>BB_width</code> , <code>BB_pos</code>	Volatilité et position dans la bande
Volatilité	<code>ATR_pct</code>	ATR normalisé par le prix
Volume	<code>Volume_ratio</code> , <code>OBV_change</code>	Pression acheteurs/vendeurs
Lags	<code>*_lag1/2/3</code> (5 features $\times$ 3)	Autocorrélation court-terme
Momentum diff.	<code>RSI_change</code> , <code>MACD_change</code> , <code>Volume_change</code> , <code>ATR_change</code>	Accélération des indicateurs

Correction v2 importante : Dans la version initiale, les SMA et EMA étaient utilisées en niveaux absolus (non stationnaires). Elles ont été remplacées par les ratios `price_vs_SMAx = (Close - SMA_x) / SMA_x`. De même, l'OBV cumulatif (non-stationnaire par nature) a été remplacé par `OBV_change`.

3.3 Split chronologique et normalisation

Le split est **strictement chronologique** (80 % entraînement, 20 % test), ce qui est indispensable pour les séries temporelles : un KFold classique mélangerait passé et futur, ce qui constitue un *data leakage* grave.

$$\text{train} = [t_0, t_{[0,8N]}] \quad \text{test} = [t_{[0,8N]+1}, t_N]$$

Un `StandardScaler` est ajusté sur le seul jeu d'entraînement, puis appliqué au test sans re-calibration (pour éviter tout leakage de statistiques futures). La normalisation n'est appliquée qu'aux modèles qui en ont besoin : Régression Logistique et SVM.

3.4 Recherche d'hyperparamètres (GridSearchCV + TimeSeriesSplit)

Choix méthodologique clé

La recherche d'hyperparamètres est effectuée **une seule fois, sur AAPL** (le ticker avec l'historique le plus long), via **GridSearchCV** avec **TimeSeriesSplit(n_splits=5)** comme schéma de validation croisée. Les paramètres retenus sont ensuite réutilisés à l'identique pour les 5 tickers.

Justification : si chaque ticker avait ses propres hyperparamètres optimisés séparément, on ne pourrait plus distinguer un effet “meilleur modèle” d'un effet “meilleur réglage”. Fixer le réglage une fois garantit une comparaison à protocole constant.

TABLE 4 – Hyperparamètres retenus par GridSearchCV (sur AAPL)

Modèle	Hyperparamètres retenus	Accuracy CV (AAPL)
Régression Logistique	$C = 10,0$	0,5083
Random Forest	$n_estimators=300, max_depth=5$	0,5160
XGBoost	$n_est.=100, lr=0.05, depth=5$	0,5210
SVM	$C = 0,1, kernel=rbf$	0,5193

Interprétation : les accuracy CV sont toutes dans l'intervalle 50,8–52,1 %, ce qui confirme que le facteur limitant est la qualité du signal disponible dans les features (propriété intrinsèque des marchés), et non le réglage des modèles. Un gain marginal post-GridSearch est lui-même un résultat informateur.

3.5 Modèles comparés

TABLE 5 – Description des quatre modèles comparés

Modèle	Principe	Justification du choix
Régression Logistique	Modèle linéaire probabiliste, régularisation ℓ_2	Baseline interprétable, détecte les relations linéaires
Random Forest	Ensemble de N arbres de décision en bagging	Robuste aux outliers, importance native des features
XGBoost	Boosting par gradient sur arbres, régularisation ℓ_1/ℓ_2	État de l'art sur données tabulaires, forte capacité
SVM	Hyperplan de séparation maximale, noyau RBF	Efficace en haute dimension, bon avec données normalisées

3.6 Métriques d'évaluation

Accuracy et **Directional Accuracy** sont mathématiquement identiques ici, puisque la cible est déjà la direction (1 = hausse, 0 = baisse). La Directional Accuracy est conservée comme colonne séparée pour répondre littéralement au sujet et pour garder la compatibilité avec d'éventuels problèmes de régression futurs (où les deux métriques divergeraient).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad \text{F1} = \frac{2 \times \text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

Le **Sharpe Ratio annualisé** mesure le rendement ajusté du risque :

$$\text{Sharpe} = \frac{\mu_r}{\sigma_r} \times \sqrt{252}$$

où μ_r est le rendement journalier moyen de la stratégie et σ_r son écart-type.

4 Résultats

4.1 Analyse détaillée — AAPL (ticker principal)

4.1.1 Benchmark des 4 modèles

TABLE 6 – Benchmark complet des 4 modèles sur AAPL

Modèle	Accuracy	F1-Score	Dir. Acc.	Temps (s)	Taille (Ko)
SVM	0,5460	0,7063	0,5460	1,26	582,3
XGBoost	0,5349	0,5757	0,5349	0,57	222,4
Régression Logistique	0,5294	—	0,5294	—	—
Random Forest	0,5276	—	0,5276	—	—

Note critique — SVM dégénéré sur AAPL et MSFT : Le SVM ($C = 0,1$, noyau RBF), très régularisé, prédit en réalité la classe “hausse” pour **100 % des jours de test** sur AAPL et MSFT. Son accuracy élevée (54,6 %) ne reflète pas une capacité de discrimination réelle : elle résulte du fait que sur un marché fortement haussier, prédire systématiquement la hausse donne mathématiquement la même performance qu'un Buy & Hold. La permutation importance confirme cette dégénérescence : perturber n'importe quelle feature ne change pas la prédiction. Ce comportement est automatiquement détecté et signalé dans l'application Streamlit.

4.1.2 Tests de significativité statistique

Test binomial : teste si l'accuracy de chaque modèle est statistiquement supérieure à 50 % (hasard pur).

TABLE 7 – Test binomial vs hasard (50 %) — AAPL

Modèle	Accuracy	p -valeur	Significatif à 5 % ?
Régression Logistique	0,5294	0,0919	Non
Random Forest	0,5276	0,1068	Non
XGBoost	0,5349	0,0563	Non
SVM	0,5460	0,0178	Oui

Interprétation

Seul le SVM passe le seuil de 5 % ($p = 0,0178$) — mais, comme établi ci-dessus, cette significativité provient de sa stratégie dégénérée (toujours prédire la hausse sur marché haussier) plutôt que d'une capacité prédictive réelle. Les trois autres modèles **ne sont pas statistiquement supérieurs au hasard** sur AAPL. Ce résultat est cohérent avec la littérature en finance quantitative : des accuracies de 50–55 % sont typiques sur des horizons journaliers, et un test binomial significatif ne garantit pas une significativité économique (après coûts de transaction).

Test de McNemar (SVM vs XGBoost, les deux meilleurs modèles) :

TABLE 8 – Test de McNemar — SVM vs XGBoost sur AAPL

Statistique	p -valeur	Différence significative ?
100,0	0,7277	Non

Le test de McNemar ($p = 0,73$) indique que SVM et XGBoost **font des erreurs sur les mêmes jours** : ils ne sont pas complémentaires, mais redondants. Cette information est utile pour écarter une stratégie d'ensemble basée sur le vote de ces deux modèles, qui n'apporterait pas de gain substantiel.

4.1.3 Importance des variables

Les variables les plus influentes, mesurées par permutation importance (métrique comparable entre toutes les familles de modèles), sont présentées dans le tableau 9.

Interprétation : les features de momentum (MACD et ses dérivés, RSI, lags) dominent systématiquement les classements. Les features de lag (J-1 à J-3) sont présentes dans tous les modèles à base d'arbres, suggérant que le signal exploitable

TABLE 9 – Top 5 features par permutation importance — AAPL

Modèle	Features les plus influentes
Régression Logistique	BB_pos, MACD_lag1, MACD_signal, MACD, price_vs_SMA50
Random Forest	BB_width, MACD_lag1, MACD, ATR_pct, Volume_ratio_lag2
XGBoost	RSI_14, MACD_signal, RSI_14_lag3, RSI_14_lag1, Volume_change
SVM	(dégénéré — aucune feature influente isolément)

est principalement de l'**autocorrélation court-terme**. Les indicateurs de position (BB_pos) et de volatilité (BB_width, ATR_pct) complètent ce signal.

L'accord entre **feature_importances_** (propre aux arbres) et la permutation importance sur Random Forest et XGBoost renforce la confiance dans ces features : l'absence de contradiction entre les deux méthodes indique que les corrélations inter-features ne biaisent pas fortement le classement natif.

4.2 Synthèse globale — 5 tickers

TABLE 10 – Synthèse globale — Meilleur modèle par ticker

Ticker	Nom	Meilleur modèle	Acc.	F1	Rdmt modèle	Rdmt B&H	Sharpe mod.
AAPL	Apple	SVM	0,546	0,706	+62,1 %	+62,1 %	0,96
MSFT	Microsoft	SVM	0,548	0,708	+47,0 %	+47,0 %	0,93
BNP.PA	BNP Paribas	XGBoost	0,500	0,533	+19,0 %	+74,3 %	0,53
BTC-USD	Bitcoin	XGBoost	0,538	0,576	+236,1 %	+156,0 %	1,43
ETH-USD	Ethereum	SVM	0,527	0,585	+8,9 %	-20,7 %	0,32

4.3 Analyse comparative inter-tickers

4.3.1 Volatilité et modèle gagnant

L'hypothèse classique selon laquelle une faible volatilité favorise les modèles linéaires et une forte volatilité favorise les modèles non-linéaires **ne se vérifie pas simplement** dans nos résultats :

- Sur **AAPL** et **MSFT** (volatilité la plus faible), le SVM gagne — mais comme démontré plus haut, ce SVM gagne *par défaut* en prédisant systématiquement la hausse sur un marché haussier, pas par discrimination réelle.
- Sur **BNP.PA** (volatilité intermédiaire), aucun modèle n'extrait de signal exploitable (accuracy exactement 0,50 pour le meilleur modèle).

- Sur **BTC-USD** (forte volatilité), XGBoost gagne et produit la seule stratégie modèle qui bat nettement le Buy & Hold.
- Sur **ETH-USD** (volatilité maximale), le SVM démontre une valeur différente : la stratégie modèle protège le capital (+8,9 %) là où le Buy & Hold est négatif (−20,7 %).

Conclusion inter-tickers

Le facteur le plus déterminant semble être le **régime de marché sur la période de test** (haussier pour AAPL/MSFT, baissier pour ETH, très haussier et volatile pour BTC) plutôt que la volatilité ou la taille du dataset seules. Cette conclusion nuancée est plus informative que de forcer une interprétation qui ne correspondrait pas aux résultats effectivement obtenus.

4.3.2 Cas remarquable : BTC-USD avec XGBoost

BTC-USD est le seul actif pour lequel la stratégie modèle bat le Buy & Hold *à la fois* en rendement absolu (236,1 % vs 156,0 %) et en Sharpe Ratio (1,43 vs 0,95). Ce résultat suggère que la forte volatilité des crypto-monnaies, si elle rend la prédiction plus difficile en termes d'accuracy absolue, peut paradoxalement générer plus d'opportunités exploitables sur des horizons journaliers : les alternances rapides de tendances créent des structures d'autocorrélation court-terme que XGBoost capte efficacement.

4.3.3 Cas remarquable : ETH-USD

Sur **ETH-USD**, le Buy & Hold aurait produit une perte de 20,7 % sur la période de test (marché baissier). La stratégie modèle, en évitant les jours de signal négatif (signal = 0), limite cette perte à une performance de +8,9 %. La valeur ajoutée ici n'est pas la sur-performance, mais la **protection du capital**, un critère distinct du rendement absolu.

4.4 Simulation de trading

4.4.1 Principe

La stratégie modèle est définie ainsi : si le modèle prédit une hausse le jour t ($\hat{y}_t = 1$), on investit la totalité du capital le jour $t + 1$ (en utilisant signal_t décalé d'un jour pour éviter tout look-ahead bias). Sinon, on reste en cash. Le rendement cumulé est comparé à celui du Buy & Hold (investi en permanence).

$$r_t^{\text{modèle}} = r_t^{\text{journalier}} \times \text{signal}_{t-1} \quad \text{Rendement cumulé} = \prod_t (1 + r_t^{\text{modèle}})$$

4.4.2 Limites de la simulation

1. **Absence de coûts de transaction** : chaque aller-retour génère des frais (spread, commission), qui réduiraient le rendement net, particulièrement pour les stratégies à fort taux de rotation.
2. **Exécution au prix de clôture** : dans la réalité, on exécute à l'ouverture du lendemain, avec risque de gap nocturne.
3. **Slippage** : pour les actifs peu liquides, l'ordre de marché peut déplacer le prix.
4. **Suroptimisation** : la simulation sur données historiques ne garantit pas des performances futures (overfitting de marché).

5 Application Streamlit

L'application Streamlit (`app.py`, 752 lignes) déploie le pipeline de prédiction dans une interface interactive de type dashboard BI. Elle est organisée en six onglets :

1. **Vue d'ensemble** : synthèse multi-tickers (accuracy, rendements, Sharpe comparé), tableau volatilité / taille du dataset / modèle gagnant, panneau méthodologie (hyperparamètres GridSearchCV) et panneau significativité statistique (tests binomial et McNemar).
2. **Prix & Indicateurs** : courbe de prix sur la période choisie, avec RSI, MACD et volume superposés.
3. **Prédictions** : signaux du modèle (hausse/baisse) superposés au prix, probabilité de hausse journalière, table des dernières prédictions, export CSV. **Avertissement automatique** si le modèle prédit un signal constant (cas SVM sur AAPL/MSFT).
4. **Simulation P&L** : évolution du capital selon la stratégie modèle vs Buy & Hold, avec Sharpe Ratio comparé.
5. **Benchmark & Significativité** : tableau des 4 modèles avec graphique et radar multi-critères, badges de significativité statistique.
6. **Variables influentes** : importance native (`feature_importances_`) pour RF et XGBoost, avec repère qualitatif sur la permutation importance.

Architecture technique

Les modèles sont chargés depuis des fichiers `.pkl` (sérialisés via `pickle`) et ne sont pas réentraînés à l'exécution. Les features sont recalculées dynamiquement via `yfinance` pour la période choisie par l'utilisateur, en utilisant exactement la même fonction `build_features` que dans le notebook — garantissant la cohérence entre entraînement et inférence. Un fichier `notebook_results.py` contient les résultats v4 figés, permettant à l'onglet "Vue d'ensemble" de fonctionner même sans les `.pkl` mis à jour.

6 Discussion et Limites

6.1 Limites inhérentes à la prédiction de marché

1. **Efficience de marché (EMH)** : sur des marchés liquides (AAPL, MSFT), l'information contenue dans les prix passés est rapidement intégrée par les participants. Les indicateurs techniques sont calculés par des millions d'acteurs, ce qui érode leur valeur prédictive.
2. **Plafond d'accuracy** : la littérature académique situe typiquement les performances de prédiction de direction journalière entre 50 et 55 %. Nos résultats (50–54,6 %) sont dans cette fourchette.
3. **Non-stationnarité des régimes** : un modèle entraîné sur un régime haussier 2015–2022 peut ne pas généraliser à un régime baissier ou latéral.
4. **Look-ahead bias potentiel** : bien que contrôlé par le split chronologique et le décalage d'un jour du signal de trading, des biais subtils (ex. : calcul du RSI sur des données futures lors du backtesting) doivent être soigneusement évités.

6.2 Limites du projet

1. **Absence de données alternatives** : sentiment de marché, données macroéconomiques, actualités (NLP), carnets d'ordres — des sources supplémentaires pourraient améliorer le signal.
2. **Features techniques uniquement** : les indicateurs techniques sont dérivés des seuls prix et volumes, sans information exogène.
3. **Horizon de prédiction** : seul l'horizon $J+1$ est étudié. Des horizons plus longs ($J+5$, $J+20$) pourraient offrir des opportunités différentes.
4. **Modèles séquentiels non testés** : LSTM, Transformer financiers (TFT — Temporal Fusion Transformer) pourraient capturer des dépendances temporelles longues non accessibles aux modèles ML classiques.

6.3 Perspectives d'amélioration

- Intégrer des features de sentiment (Twitter/Reddit financier, index de peur VIX).
- Tester des modèles de séquences (LSTM, GRU, attention) sur les mêmes features.
- Ajouter une couche de gestion du risque (stop-loss, sizing dynamique du capital).
- Étendre à des horizons multi-jours avec une approche multi-output.
- Déployer l'application en production avec une base de données persistante et des alertes automatiques.

7 Conclusion

Ce projet a permis de construire un pipeline ML complet et rigoureux pour la prédiction de tendances boursières sur cinq actifs de natures différentes. Les principaux apports par rapport à une approche naïve sont :

- L'**ingénierie de features stationnaires**, qui garantit la cohérence temporelle du signal sur des séries de 10 ans.
- La **validation croisée temporelle** (TimeSeriesSplit), qui évite le data leakage et produit des estimations réalistes de performance.
- La **recherche d'hyperparamètres unifiée** (sur AAPL, réutilisée pour tous), qui garantit une comparaison équitable inter-tickers.
- Les **tests de significativité statistique** (binomial et McNemar), qui distinguent la performance réelle du bruit aléatoire.
- L'**honnêteté analytique** : identifier que le SVM "gagne" par dégénérescence sur marchés haussiers, que BNP Paribas n'offre aucun signal, et que BTC-USD est le seul cas de sur-performance réelle vs Buy & Hold.

La conclusion principale est que **la prédiction de direction journalière de marchés financiers est intrinsèquement difficile** (50–55 % d'accuracy), que les modèles ML peuvent extraire un signal modeste sur certains actifs (BTC-USD, ETH-USD), et que la valeur ajoutée réelle dépend fortement du régime de marché et non uniquement des propriétés statistiques de l'actif. Ces résultats, bien que modestes en termes absolus, constituent une contribution solide et honnête à la question posée.

A Structure du projet

Listing 2 – Arborescence du projet livré

```
1 Proj et_IA_v2/
```



```
2  Projet_IA.pdf                                # Sujet du projet
3  projet_IA_groupe5_v4.ipynb                  # Notebook principal
   (v4)
4  requirements.txt                            # Dépendances
5  streamlit_app_v4/
6      streamlit_app/
7          app.py                              # Application Streamlit
   (752 lignes)
8          notebook_results.py                 # Résultats v4 figés
9          requirements.txt
10         summary.csv                         # Synthèse globale
11         README.md
12         models/
13             AAPL.pkl
14             MSFT.pkl
15             BNP_PA.pkl
16             BTC-USD.pkl
17             ETH-USD.pkl
```

B Dépendances Python

Listing 3 – requirements.txt

```
1 streamlit>=1.30
2 yfinance>=0.2.40
3 pandas>=2.0
4 numpy>=1.24
5 scikit-learn>=1.3
6 xgboost>=2.0
7 plotly>=5.18
```

C Lancement de l'application

Listing 4 – Installation et lancement

```
1 # Installation des dependances
2 pip install -r requirements.txt
3
4 # Lancement
5 cd streamlit_app_v4/streamlit_app
6 streamlit run app.py
```

Note : Les fichiers `.pk1` fournis sont ceux de la v3. Pour refléter les résultats v4 (avec GridSearchCV), il faut régénérer les modèles depuis le notebook v4 exécuté sur Google Colab (section 6, “Export des objets pour Streamlit”), puis remplacer les fichiers `models/*.pk1`. L’onglet “Vue d’ensemble” fonctionne néanmoins avec les résultats figés dans `notebook_results.py`.

D Versions des corrections apportées

TABLE 11 – Historique des corrections (v2 et v4)

Problème initial	Correction apportée
<i>Version 2</i>	
SMA/EMA en niveaux absolus	Ratios normalisés (<code>price_vs_SMAx</code>)
OBV cumulatif non-stationnaire	Remplacement par <code>OBV_change</code>
<code>classification_report</code> non affiché	Affichage complet ajouté
Directional Accuracy absente	Ajoutée au benchmark
Courbes d’apprentissage absentes	Ajout avec <code>TimeSeriesSplit</code>
Sharpe Ratio absent	Ajout pour les deux stratégies
<i>Version 4</i>	
Hyperparamètres fixés arbitrairement	GridSearchCV + <code>TimeSeriesSplit</code> (section 3.4bis)
Dir. Acc. = Accuracy sans commentaire	Note méthodologique explicite
Courbes d’apprentissage non interprétées	Cellules markdown d’interprétation ajoutées
<code>feature_importances_</code> non comparable	Ajout de la permutation importance
Aucune analyse inter-tickers	Section 5.2 (volatilité / modèle gagnant)
Aucun test de significativité	Test binomial + McNemar (section 3.9bis)